

Ejercicios de Retrieval-Augmented Generation

Big Data Processing II

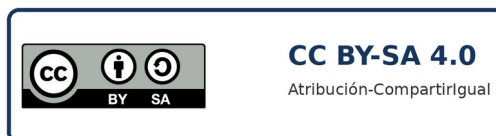
Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

RAG aplicado al análisis deportivo

Se plantean diferentes ejercicios basados en técnicas simples de RAG.

Objetivo: comprender RAG a nivel de arquitectura y construir un RAG funcional en Python con LangChain.

Requisitos y entorno

Los ejercicios prácticos están pensados para ejecutarse en un entorno Python 3.10+.

Instalación mínima (elige UNA opción de LLM):

Opción A: OpenAI (requiere OPENAI_API_KEY)

```
pip install -U langchain langchain-core langchain-community langchain-openai langchain-chroma chromadb tiktoken
```

Opción B: Ollama local (sin API key, requiere Ollama instalado)

```
pip install -U langchain langchain-core langchain-community langchain-ollama langchain-chroma chromadb tiktoken
```

Embeddings locales recomendados (sin token HF)

```
pip install -U langchain-huggingface sentence-transformers
```

Se permite usar cualquier corpus pequeño (p.ej., 2–5 documentos TXT/PDF), pero se recomienda preparar un directorio ./data con 3–6 ficheros (se proporcionan algunos de base sobre asuntos deportivos pero se pueden ampliar con los que se deseen).

Bloque A — Teoría (sin código)

Ejercicio A1 — RAG vs Fine-Tuning vs Prompting

Para un asistente sobre normativa deportiva interna (documentos privados que cambian cada semana): (1) ¿Cuándo elegirías RAG? (2) ¿Cuándo elegirías fine-tuning? (3) ¿Qué papel juega el prompt engineering en ambos? Da una respuesta razonada (máx. 12 líneas).

Ejercicio A2 — Evaluación de RAG

Propón un plan de evaluación con al menos 3 métricas o criterios: (i) métricas de retrieval (p.ej., Recall@k), (ii) métricas de respuesta (p.ej., exactitud/fidelidad), (iii) métricas de robustez (p.ej., sensibilidad a perturbaciones). Indica cómo obtendrías datos de evaluación (conjunto de preguntas, gold answers, etc.).

Bloque B — Práctica (Python + LangChain)

Ejercicio B1 — RAG mínimo funcional (TXT → Chroma → QA)

Construye un RAG mínimo que:

- 1) Cargue varios .txt desde ./data
- 2) Divida en chunks con RecursiveCharacterTextSplitter
- 3) Genere embeddings (HuggingFaceEmbeddings recomendado)
- 4) Indexe en Chroma (persistente)
- 5) Responda una pregunta con create_retrieval_chain
- 6) Realiza prompts que se creen resúmenes generales y por temáticas del RAG.

Requisitos:

- El LLM debe responder SOLO con base en el contexto recuperado.
- La respuesta debe incluir una sección 'Fuentes' con los metadatos (p.ej., filename) de los chunks usados.

Entrega: fragmento de código + breve análisis (10–12 líneas).

Entrega

Entrega un repositorio o un zip con:

- Notebook o script para el ejercicio práctico.
- Un breve README con instrucciones de ejecución.
- Respuestas a los ejercicios teóricos.

MATERIAL DOCENTE EN ABIERTO